

Waseet Logistics Capstone Project

End-to-End Pipeline Orchestration & Historical Analytics Report

Section 1: Schema Architecture, Grain, and Design Decisions

The core transactional entity in the warehouse is `parcel_scans`, with each record defined at the grain of an individual physical scan event occurring at a distinct timestamp. Unlike the Najm reference schema which treated `scan_id` as an assumed surrogate primary key, initial system testing revealed that upstream supplier re-transmissions routinely emit duplicate `scan_id` values for identical real-world events. To prevent duplicate data while allowing a single package to traverse multiple valid tracking points across transit hubs, table uniqueness is enforced through a composite constraint on (`parcel_id`, `scan_type`, `scanned_at`). The `courier_id` attribute is designed as nullable to reflect the logistics reality that initial intake and depot-to-depot line hauls occur before a parcel is assigned to a final-mile courier route. Foreign keys link records strictly to dimensional tables for hubs, `service_levels`, and customers. A composite B-tree index on (`parcel_id`, `scanned_at`) supports rapid parcel tracking lookups and accelerates range deletions required during idempotent reloads. An audit ledger table, `load_log`, captures pipeline execution telemetry, storing execution dates, row counts (read, loaded, rejected, and deduplicated), terminal execution states, and verbatim error traces for auditability.

Section 2: Phase-0 Data Profiling and Anomaly Quantifications

Exploratory profiling of the raw daily CSV files identified six consistent data quality issues requiring deterministic remediation. First, duplicate scan events occurred intermittently in routine files—such as 3 duplicate scans on May 4—and culminated in an unannounced batch re-transmission on May 13 containing 1,298 duplicate rows. Second, timestamp formats exhibited systematic schema drift, with roughly 3.2% of entries recorded as DD/MM/YYYY HH:MM instead of the expected ISO-8601 standard (YYYY-MM-DD HH:MM:SS), necessitating multi-pattern timestamp parsing. Third, approximately 1.5% of parcel weights used European comma-decimal formatting (e.g., "12,5"), mixed with occasional non-numeric strings and extreme outliers outside the plausible operational range. Fourth, the `scan_type` attribute contained irregular casing (such as "pickup" and "Delivered"), requiring uppercase normalization before warehouse ingestion. Fifth, foreign key audits revealed invalid depot entries referencing `hub_id` 99, which has no matching dimension record. Sixth, customer reference checks identified missing customer profiles, requiring automated backfill from the external customer REST API.

Section 3: The Declarative Quality Suite

Data quality validation is implemented as a declarative suite of ten rules executed against inbound raw CSV files rather than embedded as procedural filtering masks within transform logic. These rules enforce structural completeness, timestamp and formatting validity, categorical set containment, and numerical boundaries. Specifically, the suite validates not-null constraints on `scan_id`, `parcel_id`, and `customer_id`; verifies ISO timestamp syntax; restricts `scan_type` to approved transit states; limits `hub_id` to the integer range 1 through 10; and bounds `weight_kg` between 0.01 kg and 100.0 kg. Schema drift detection operates against a version-controlled, static `baseline_schema.json` to catch header changes before processing begins. When unmapped columns, missing attributes, or quality threshold breaches occur, the suite exits with a non-zero code (`sys.exit(1)`), deliberately failing the Airflow quality task to prevent corrupted data from settling into production tables.

Section 4: The 21-Day Backfill Reconciliation and Special Operational Incidents

Executing the full 21-day historical backfill confirmed pipeline idempotency and operational auditability, with every day satisfying the balance identity: `rows_read equals rows_loaded plus rows_rejected plus rows_deduplicated`. Four operational edge cases tested the pipeline's failure handling:

- On May 10, the upstream supplier omitted the daily file upload entirely. The Airflow FileSensor polled in reschedule mode, reached its 30-second timeout threshold, and triggered the `alert_failure` callback without creating partial or orphaned database records.
- On May 13, an unannounced 2,546-row delivery arrived containing an accidental duplicate concatenation. The pipeline extracted the payload, deduplicated 1,298 redundant events, quarantined invalid entries, and loaded exactly 1,248 clean rows, closing the day with zero reconciliation variance.
- On May 15 (Eid Holiday), the delivery network closed and transmitted a header-only file with zero data rows. The BranchPythonOperator identified the empty file and routed execution to `skip_day`, allowing the DAG run to finish successfully under trigger rule `none_failed_min_one_success` without triggering false failure alarms.
- On May 19, supplier schema drift renamed the `weight_kg` column. The fail-fast contract check detected the missing column prior to transformation, aborted execution with exit code 1, and logged the contract failure to `load_log`.

Across all standard operational dates—such as May 1 (1,230 read, 1,208 loaded, 22 quarantined) and May 21 (1,222 read, 1,200 loaded, 22 quarantined)—reconciliation balanced completely.

Section 5: Next Steps and Remaining Operational Vulnerabilities

While the current architecture ensures transactional consistency and guards against known supplier defects, three operational vulnerabilities remain for production hardening. First, although invalid records are isolated in daily quarantine CSV files, there is no automated Dead Letter

Queue (DLQ) replay pipeline; re-processing corrected records currently requires manual script execution. Second, the customer API ingestion runs sequentially, leaving downstream reverse-ETL processes susceptible to upstream network bottlenecks during high-volume customer registrations. Moving customer ingestion to an asynchronous, worker-based task queue would improve throughput. Third, while the PySpark batch job properly uses broadcast hash joins for small dimension tables (hubs and service_levels), partitioning an 18 MB historical dataset across 334 distinct daily directories produced hundreds of tiny Parquet files averaging 15 to 50 KB. In an enterprise lakehouse environment, this partition scheme should be consolidated into monthly intervals (yyyy-MM) to achieve standard 128 MB to 512 MB file chunks, reducing storage metadata overhead and boosting query performance.